

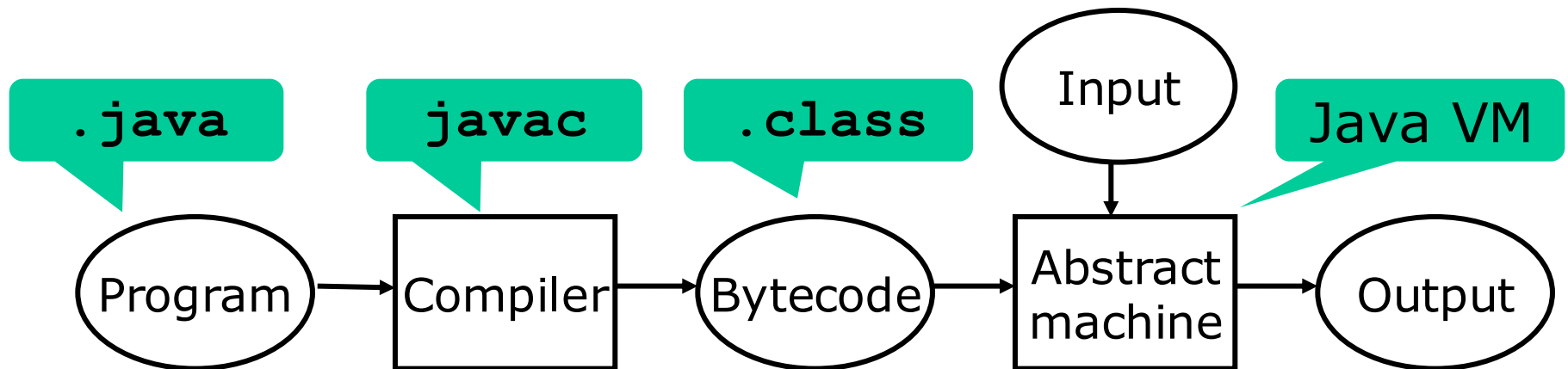
Programming Language Concepts Garbage Collection

Peter Sestoft
Niels Hallenberg
2026-07-24



Today

- Garbage collection (GC) techniques
 - Reference-counting
 - Mark-sweep
 - Two-space stop and copy
 - The garbage collectors in JVM and .NET
- List-C, a version of Micro-C with a heap and GC



Predictable Lifetime (Stack)

- By design of micro-C, stack allocation is possible:
 - Static (or lexical) scope rules. Lifetime of a variable is determined by the program text only.
 - Nested scope, i.e., blocks within blocks.
 - Function values are not returned from functions, i.e., no need for closures.
 - No dynamic data structures allowed. All data has known size, including arrays.
- Without type check, we can force a dangling pointer situation, see `exGC01.lc`

```
void main() {  
    int *ip;  
    ip = f();  
    print *ip; // 42  
    g();  
    print *ip; // 9999  
}
```

```
int f() {  
    int i;  
    i = 42;  
    return &i;  
}
```

```
void g() {  
    int i;  
    i = 9999;  
}
```



Unpredictable Lifetime (Heap)

- Modern programming languages permit unpredictable lifetimes of values:
 - Functions as values, i.e., closures.
 - Dynamic data structures, e.g., lists, and trees.
 - They have objects
 - Etc.
- Some languages require explicit allocation and de-allocation of heap values, i.e., C and C++.
 - In C you write
 - `int *ia = (int *)malloc(len*sizeof(int));` to allocate an array of `len` integers.
 - `free(ia);` to de-allocate the array again.
- Some languages have automatic allocation and de-allocation, e.g., Java, C#, F# etc.



Java program (ex6java.java)

```
class Node extends Object {  
    Node next;  
    Node prev;  
    int item;  
}
```

Double linked list
implementation.

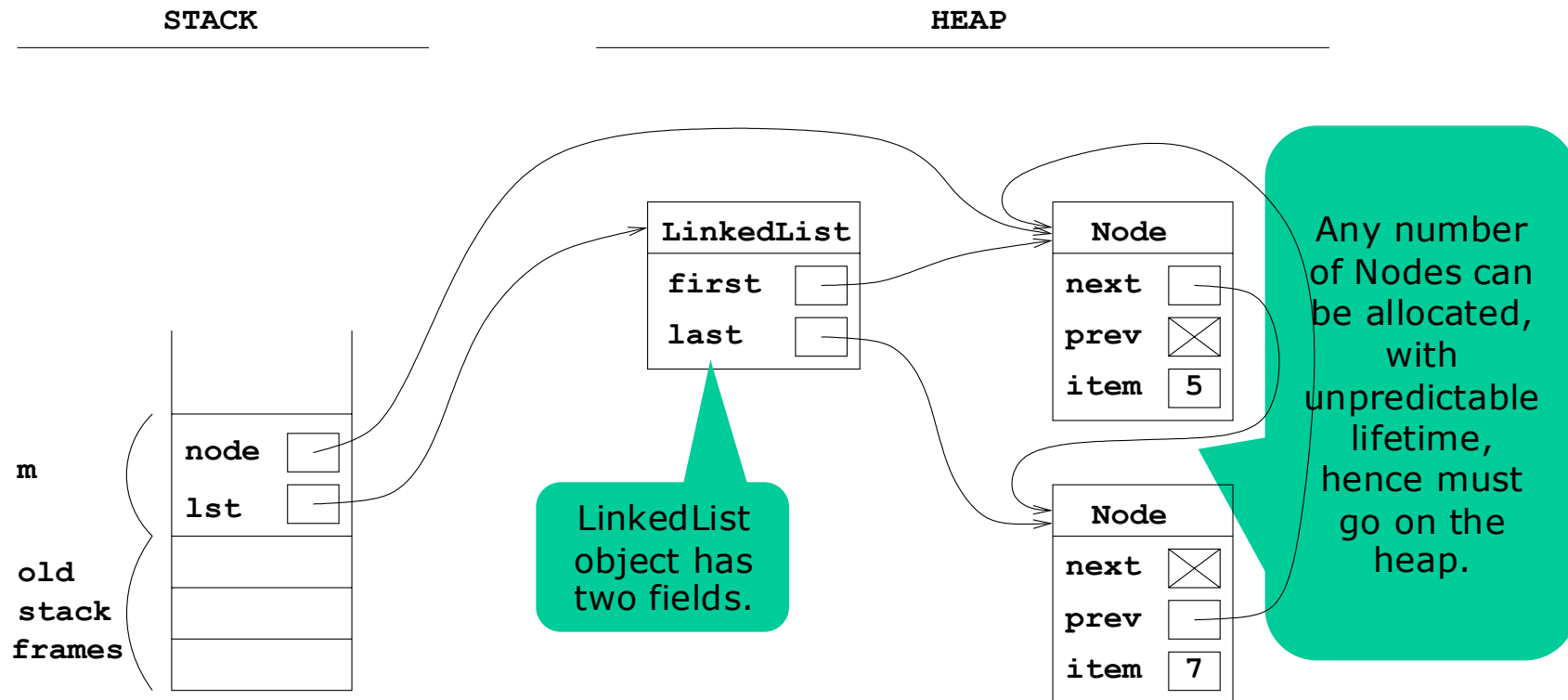
Ex6java.java
in folder
Virtual/Ex6.

```
class LinkedList extends Object {  
    Node first, last;  
  
    void addLast(int item) {  
        Node node = new Node();  
        node.item = item;  
        if (this.last == null) {  
            this.first = node;  
            this.last = node;  
        } else {  
            this.last.next = node;  
            node.prev = this.last;  
            this.last = node;  
        }  
    }  
  
    void printForwards() { ... }  
    void printBackwards() { ... }  
}
```



Example allocating in the heap

```
void m() {  
    LinkedList lst = new LinkedList();  
    lst.addLast(5);  
    lst.addLast(7);  
    Node node = lst.first;  
}
```

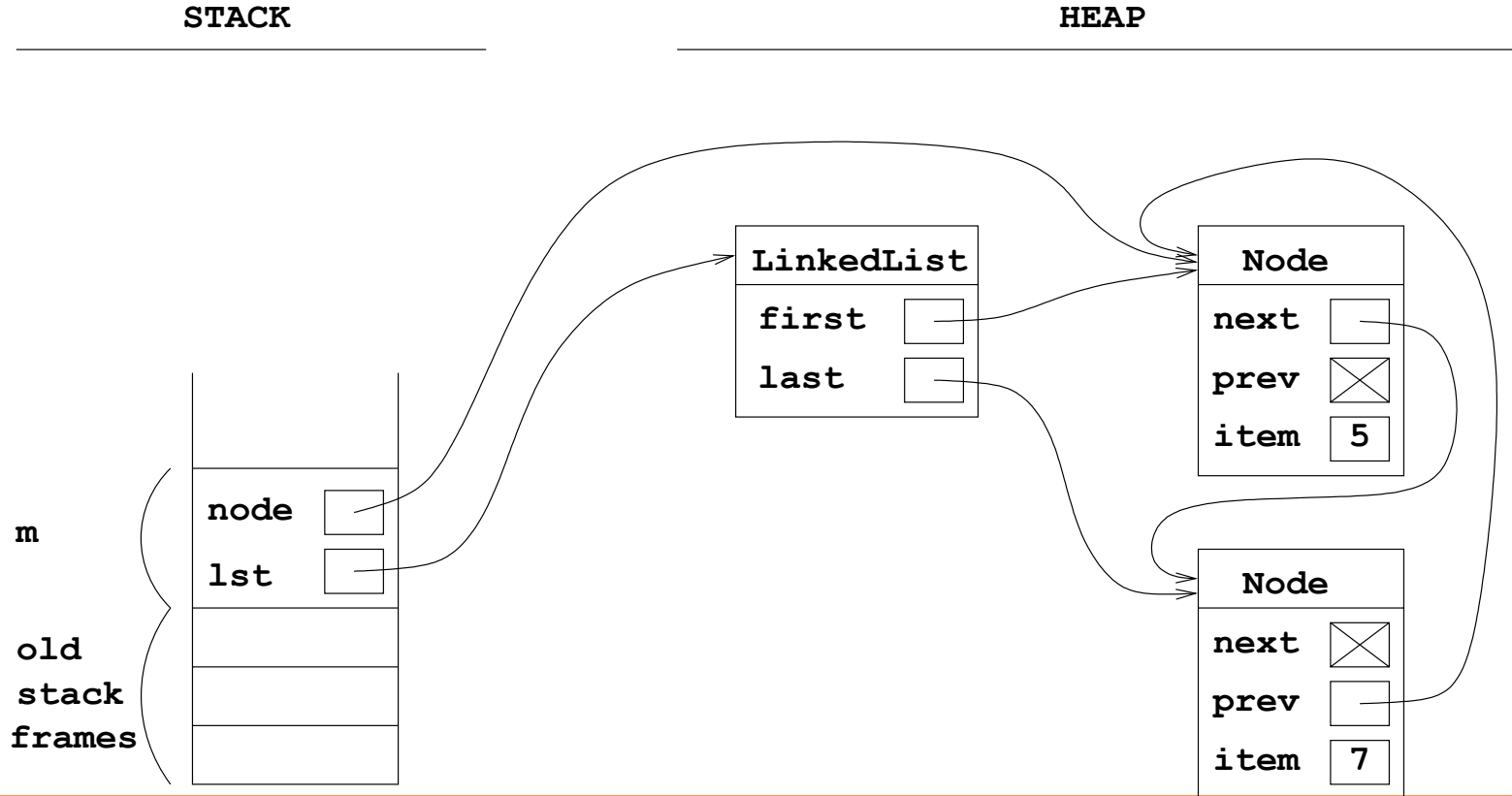


Garbage collection

- The **Mutator** is the component allocating and updating the heap, e.g., the abstract machine.
- The **Collector** is the component that reclaims unused memory, e.g., the garbage collector.
- The **Root** set is the set of live variables. The transitive closure found given the root set defines the set of active data. Data, not part of the transitive closure, is dead.
- Kinds of garbage collectors:
 - A: Reference counting
 - B: Mark-sweep
 - C: Two-space stop-and-copy, compacting
 - D: Generational
 - E: Conservative

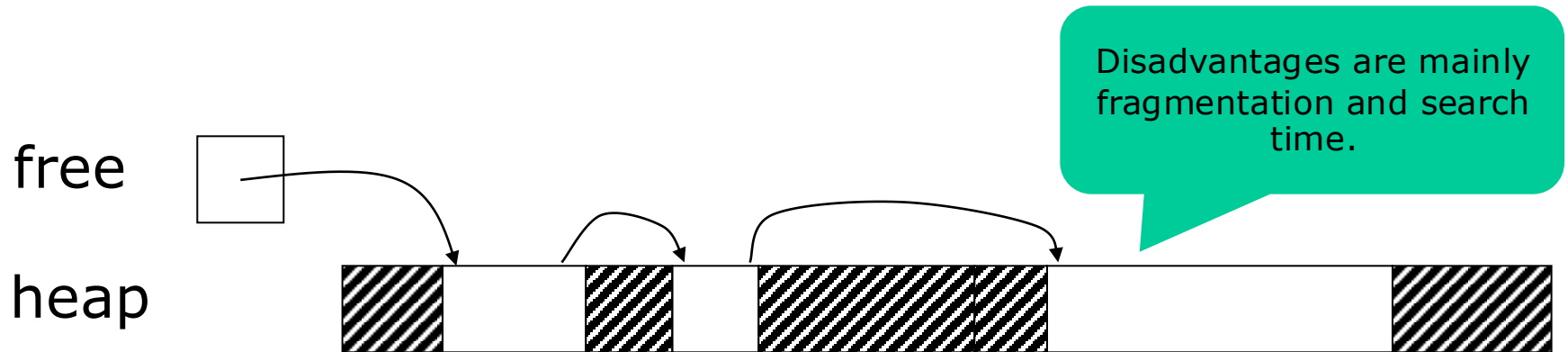
The heap as a graph

- The heap is a *graph*: *node*=object, *edge*=reference
- An object is *live* if reachable from *roots*
- Garbage collection *roots* = stack elements

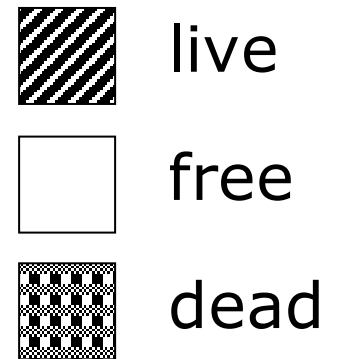


The freelist

- A freelist is a linked list of free heap blocks:



- Allocation from freelist:
 - Search for a large enough free block
 - If none found, do garbage collection
 - Try the search again
 - If it fails, we are out of memory



A: Reference counting with freelist

- Each object knows the number of references to it
- Allocate objects from the freelist
- After assignment $x=o$; the runtime system
 - Increments the count of object o
 - Decrements the count of x 's old reference (if any)
 - If that count becomes zero,
 - put that object on the freelist
 - recursively decrement count of all objects it points to
- Good
 - Simple to implement
- Bad
 - Reference count field takes space in every object
 - Reference count updates and checks take time
 - A cascade of decrements takes long time, gives long pause
 - Cannot de-allocate cyclic structures

B: Mark-sweep with freelist

- Allocate objects from the freelist
- GC phase 1: mark phase
 - Assume all objects are white to begin with
 - Find all objects that are reachable from the stack, and color them black
- GC phase 2: sweep phase
 - Scan entire heap, put all white objects on the freelist, and color black objects white
- Good
 - Rather simple to implement
- Bad
 - Sweep must look at entire heap, also dead objects; inefficient when many small objects die young
 - Risk of *heap fragmentation*

C: Two-space stop and copy

- Divide heap into to-space and from-space
- Allocate objects in from-space
- When full, recursively move all reachable objects from from-space to the empty to-space
- Swap (empty) from-space with to-space
- Good
 - Need only to look at live objects
 - Good reference locality and cache behavior
 - Compacts the live objects: no fragmentation
- Bad
 - Uses twice as much memory as maximal live object size
 - Needs to update references when moving objects
 - Moving a large object (e.g., an array) is slow
 - Very slow (much copying) when heap is nearly full



D: Generational garbage collection

- Observation: Most objects die young
- Divide heap into *young* (nursery) and *old* generation
- Allocate in young generation
- When full, move live objects to old gen. (minor GC)
- When old gen. full, perform a (major) GC there
- Good
 - Recovers much garbage fast
- Bad
 - May suffer fragmentation of old generation (if mark-sweep)
 - Needs a write barrier test on field assignments:
After assignment $o.f=y$ where o in old and y in young,
need to remember that y is live

In what language, C#, Java or F#, is the write barrier challenge most problematic and why?

E:Conservative garbage collectors

- Is 0xFFFFFFFFFA on the stack an int or a heap ref?
- If the GC doesn't know, it must be *conservative*: Assume it could be a reference to an object.
- Conservative collectors exist as C/C++ libraries
- Good
 - Can be added to C and C++ programs as a library
 - Works even with pointer arithmetic's
- Bad
 - Unpredictable memory leaks
 - Cannot be compacting updating a “reference” that is a customer number leads to madness



Concurrent garbage collection

- In a multi-kernel machine, let one kernel run GC
- Complicated
 - Race conditions when allocating objects
 - Race conditions when moving objects
- Typically suspends threads at "GC safe" points
 - May considerably reduce concurrency (because one thread may take long to reach a safe point)
- Or introduce load and store barriers.

GC in mainstream virtual machines

- Sun/Oracle Hotspot JVM (client+server)
 - Three generations
 - When gen. 0 is full, move live objects to gen. 1
 - Gen. 1 uses two-space stop-and-copy GC; when objects get old, they are moved to gen. 2
 - Gen. 2 uses mark-sweep with compaction
- IBM JVM (used in e.g., WebSphere server)
 - Highly concurrent generational; see David Bacon's paper
- Microsoft .NET (desktop+server)
 - Three generation small-obj heap + large-obj heap
 - When gen. 0 is full, move to gen. 1
 - When gen. 1 is full, move to gen. 2
 - Gen. 2 uses mark-sweep with occasional compaction
- Mono .NET implementation
 - Two-generational (stop-and-copy plus M-S)
- Java 21: GC G1, ZGC, Generational ZGC.



Other GC-related topics

- *Large object space*: Large arrays and other long-lived objects may be stored separately
- *Weak reference*: A reference that cannot itself keep an object live
- *Finalizer*: Code that will be executed when an object dies and gets collected (e.g., close file)
- *Resurrection*: A finalizer may make a dead object live again (yrk!)
- *Pinning*: When Java/C# exports a reference to C/C++ code, the object must be pinned; if GC moves it, the reference will be wrong

GC stress (StringConcatSpeed.java)

- What do these loops do? Which is better?

```
StringBuilder buf  
    = new StringBuilder();  
for (int i=0; i<n; i++)  
    buf.append(ss[i]);  
res = buf.toString();
```

```
String res = "";  
for (int i=0; i<n; i++)  
    res += ss[i];
```

```
javac StringConcatSpeed.java  
java -verbosegc StringConcatSpeed  
Found in folder  
Virtual/StringConcatSpeed.
```

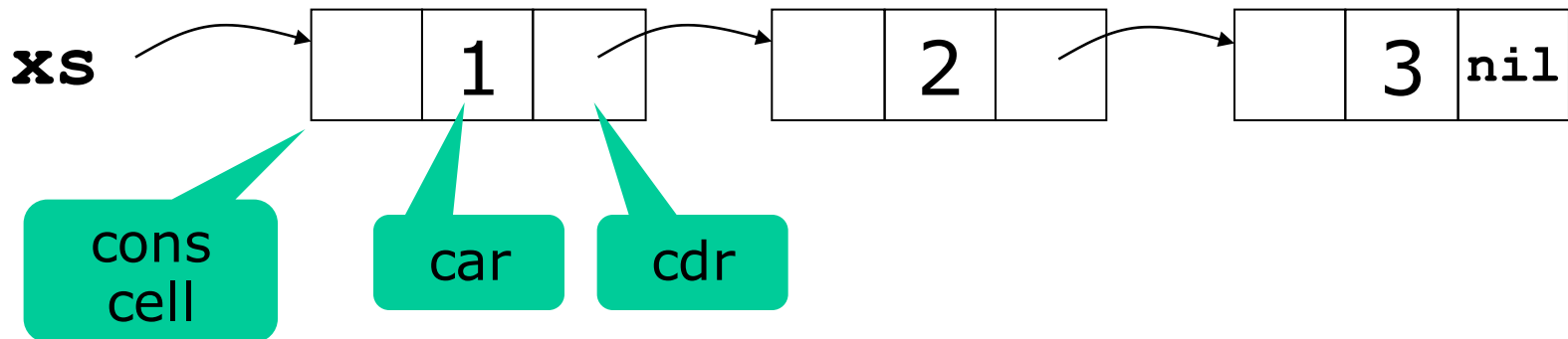
New: List-C and the list machine

- list-c = micro-C with Lisp/Scheme data

```
void main(int n) {  
    dynamic xs;  
    xs = nil;  
    while (n>0) {  
        xs = cons(n,xs);  
        n = n - 1;  
    }  
    printlist(xs);  
}
```

```
void printlist(dynamic xs) {  
    while (xs) {  
        print car(xs);  
        xs = cdr(xs);  
    }  
}
```

ex31.lc



A few List-C programs

The ListC folder includes some example programs:

- `ex30.lc` : Allocates cons cells that immediately dies.
- `ex31.lc`: Allocates cons cells that are live throughout the computation. Adds in front of the list.
- `ex32.lc`: Keeps a big list live throughout GC. A recursive implementation of the mark phase in a mark & sweep allocator may run out of stack space.
- `ex33.lc`: Various list manipulations: sum, append, reverse.
- `ex34.lc`: Accessing components, i.e., car and cdr in a cons cell.
- `ex35.lc`: Sharing the same cons cell between two components of another cons cell; to verify the implementation of a copying collector.
- `ex36.lc`: Creating a cyclic structure used to check that a collector does not go into an infinite loop.

List machine instructions

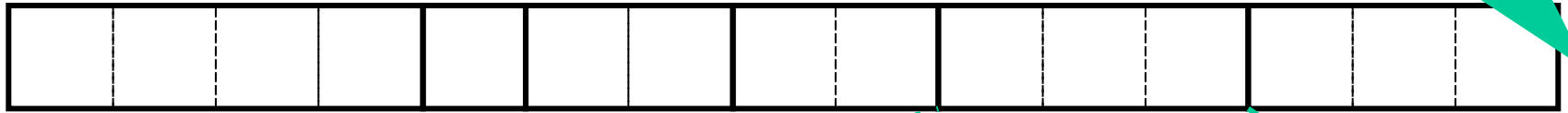
- List machine = micro-C abstract machine plus six extra instructions:
 - NIL: Put nil reference on stack
 - CONS: Allocate two-word block on heap, put reference to it on stack
 - CAR, CDR: Access word 1 or 2 of block
 - SETCAR, SETCDR: Set word 1 or 2 of block

Instruction		Stack before	Stack after	Effect
1	NIL	s	$\Rightarrow s, nil$	Load <i>nil</i> reference
110	CONS	s, v_1, v_2	$\Rightarrow s, p$	Create cons cell $p \mapsto (v_1, v_2)$ in heap
111	CAR	s, p	$\Rightarrow s, v_1$	Component 1 of $p \mapsto (v_1, v_2)$ in heap
112	CDR	s, p	$\Rightarrow s, v_2$	Component 2 of $p \mapsto (v_1, v_2)$ in heap
113	SETCAR	s, p, v	$\Rightarrow s, p$	Set component 1 of $p \mapsto _$ in heap
114	SETCDR	s, p, v	$\Rightarrow s, p$	Set component 2 of $p \mapsto _$ in heap

The structure of the list machine heap

- The heap consists of 64-bit (8-byte) *words*
- The heap is covered by *blocks*

Variable
afterheap
points at first
cell after the
heap.



Variable `heap`
points at first
cell of the
heap.



64 bits

Block tag,
always 0

Block length,
here 2

GC
color

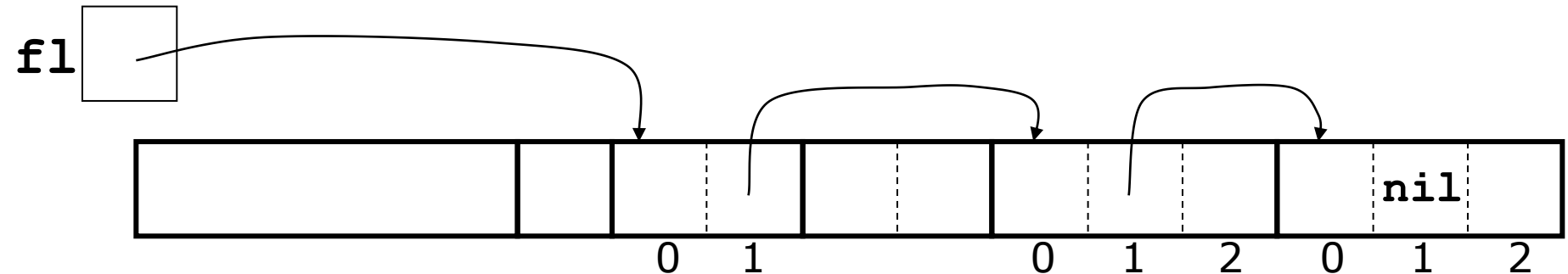
Garbage collection bits gg

Bits	Color	Meaning
00	white	After mark phase: Not reachable from stack; may be collected
01	grey	During mark phase: Reachable, referred-to blocks not yet marked
10	black	After mark phase: Reachable from stack; cannot be collected
11	blue	On freelist, or is orphan block

- The *mark phase* paints all reachable blocks black
- The *sweep phase* paints black blocks white; paints white blocks blue and puts them on freelist

The freelist; orphans

- All blocks on the freelist are blue (gg=11)
- Second word contains a reference to the next freelist element, or nil:



- A block of length zero is an *orphan*
- It consists of a header only
- (Created by allocating almost all of a block)
- Cannot be on freelist: no room for next ref.

Distinguishing integers and references

- For *exact* garbage collection we need to distinguish integers from references
- Old trick:
 - Make all heap blocks begin on address that is a multiple of 8, binary address has form xxxxxxxxxxxxxx000
 - Represent integer n as $2n+1$, so the integer's representation has form xxxxxxxx1
- Test for IsInt(v): $(v) \& 1 == 1$
- Tagging an int: $((v) \ll 1) | 1$
- Untagging an int: $(v) \gg 1$

List machine implementation

- Found in file `listmachine.c`
- Compiled on Unix with, see README.TXT for Windows.
 - `gcc listmachine.c -o listmachine`
- Execution: `./listmachine -trace ex30.out 8`
- Functions related to allocation and GC:
 - `allocate(...)`: allocates from the freelist.
 - `markPhase()`: your implementation of marking.
 - `sweepPhase()`: your implementation of sweeping.
 - `initheap()`: allocate initial heap and put in freelist.
 - `heapStatistics()`: for statistics, and to make sure the heap and freelist seems ok.

An example list-C program, ex30.lc

- Each iteration allocates a cons cell that dies
- Without a garbage collector the program soon runs out of memory

```
void main(int n) {  
    dynamic xs;  
    while (n>0) {  
        xs = cons(n, 22);  
        print car(xs);  
        n = n - 1;  
    }  
}
```

Assignment
causes previous
xs value to die

Allocate
cons cell
in heap

- Your task in BOSC: Implement garbage collectors: mark-sweep, and stop-and-copy

Extension of the compiler

The `Comp.fs` file has been extended in these places:

- `allocate`: To allocate cons cells of type `dynamic`.
- `cExpr`: To compile expressions: `NIL` constant, the unary primitives `car` and `cdr`, the binary primitives `cons`, `setcar` and `setcdr`.

Then of course the lexer and parser specifications have been modified.

The machine specification, `Machine.fs` has also been extended with the new stack instructions.